Internet Archive

Proposal - Google Summer of Code 2026

Personal Details

- Name : Ritesh hooda
- Github link: https://github.com/Ritesh-251
- Linkedin : https://www.linkedin.com/in/ritesh-hooda-010b7831b/
- Gmail: hareradheharekrishna23@gmail.com
- Country of Residence : India
- Timezone : IST (India)
- Language : English

Project Proposal

Project Title

Wayback Machine PDF Diff Engine with Analyzer-Guided Adaptive Extraction

Overview

The Wayback Machine enables comparison of archived web pages through a structured diff pipeline. However, this capability is currently limited to HTML and does not extend to PDFs, which are widely used for official documents such as policies, reports, and research publications.

This project proposes a backend PDF diff engine that integrates with the existing web-monitoring-difference system and produces JSON output fully compatible with the wayback-diff frontend.
Unlike HTML, PDFs lack a semantic structure and instead encode content as positioned elements. Therefore, meaningful comparison requires reconstructing structure from layout.

The system will use an **analyzer-guided adaptive extraction pipeline**, where extraction strategies are selected based on measurable document characteristics rather than fixed or trial-based approaches.

Problem Statement

The Wayback Machine currently enables meaningful comparison of HTML snapshots through a structured diffing pipeline. However, this capability does not extend to PDF documents, which are commonly used for publishing critical information such as policies, reports, and legal records.

The primary challenge is not simply computing differences between two PDFs, but transforming them into a representation that allows meaningful comparison. Unlike HTML, PDFs do not provide a semantic structure such as a DOM. Instead, they encode content as positioned text and graphics, making extraction inconsistent and highly dependent on document layout.

As a result, naive text extraction often introduces noise such as broken line structures, misplaced content ordering, and repeated layout artifacts. These issues lead to false positives and reduce the usefulness of diff outputs.

Therefore, the core problem is to design a system that can:

- extract content reliably across diverse PDF formats
- normalize structural noise to preserve semantic meaning
- generate diff outputs that are consistent with the Wayback diff pipeline

System Understanding

The Wayback Machine diff system follows a clear request–response pipeline involving a frontend rendering layer and a backend diff computation service.

Frontend (wayback-diff):

The wayback-diff frontend is responsible for user interaction and visualization:

- Allows users to select two archived snapshots of a URL
- Constructs a diff request containing both snapshot identifiers
- Sends the request to the backend diff service
- Receives a structured JSON response
- Renders the differences (additions, deletions, modifications) visually

Importantly, the frontend does not depend on the content type (HTML, PDF, etc.).
It only relies on the structure of the JSON response to render the diff.

Backend (web-monitoring-diff):

The web-monitoring-diff backend handles the core diff computation:

- Retrieves archived content for the selected snapshots
- Processes the content (currently optimized for HTML)
- Computes differences using a structured diff pipeline
- Returns a standardized JSON output describing the changes

This JSON includes tokens, change types, and positional information required by the frontend.

Integration Insight for PDF Support

A key architectural advantage is that the frontend is backend-agnostic and depends only on the JSON schema.

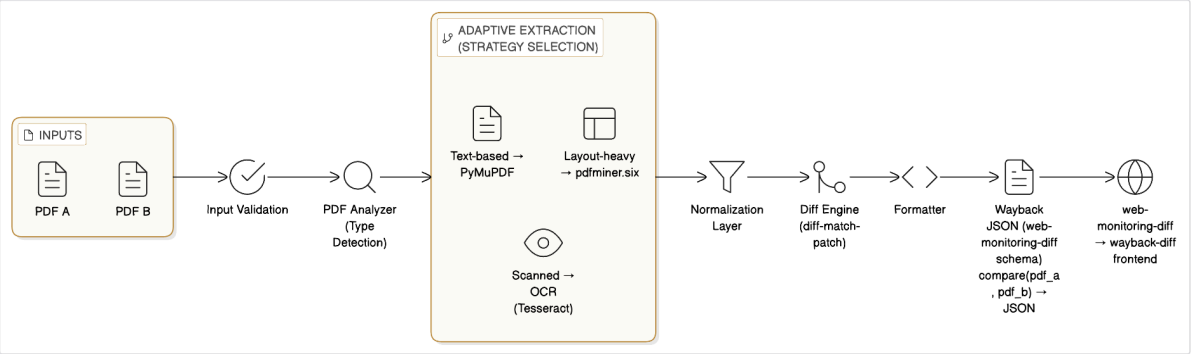
This enables a clean extension for PDF support:

- The backend can detect the content type using the MIME type (application/pdf)
- Based on this, it can route the request to a PDF-specific diff engine
- The PDF diff engine must produce output in the **same JSON schema** as the HTML diff system

Because the output format remains consistent:
No changes are required in the frontend.The PDF diff engine becomes a drop-in backend extension

Proposed Solution

Core Approach



I propose an **analyzer-guided adaptive PDF diff pipeline** that selects extraction strategies based on document characteristics and produces Wayback-compatible JSON output.

PDF Analyzer

The PDF Analyzer is a lightweight decision layer that determines the most suitable extraction strategy using measurable heuristics rather than static rules.

It evaluates:

- text density (characters per page)
- extraction success rate
- fragmentation score (irregular token distribution)
- layout complexity (based on x-coordinate clustering)

Based on these signals:

- low text density → OCR (Tesseract)
- high layout complexity → pdfminer.six
- otherwise → PyMuPDF

This ensures extraction is both adaptive to document characteristics and deterministic in behavior.

Adaptive Extraction Layer

The extraction layer executes the strategy selected by the analyzer, ensuring consistent and predictable behavior across different document types.
All extraction outputs are normalized into a unified intermediate format to standardize downstream processing.

Normalization Layer

To reduce noise and improve diff quality:

- Normalize whitespace and line breaks
- Merge hyphenated words
- Remove repeated headers and footers
- Collapse formatting artifacts
- Clean encoding inconsistencies

Representation Strategy

Instead of treating PDFs as flat text, the system will convert documents into a structured intermediate representation consisting of text blocks with positional metadata (bounding boxes).

Each block represents a semantically grouped unit (e.g., paragraph, heading, or table cell approximation). Blocks will be ordered using spatial heuristics:

- top-to-bottom, left-to-right ordering
- column detection using x-coordinate clustering

This representation reduces layout-induced noise and allows diff computation at a block level rather than raw text.

Diff Engine

The diff process operates in two stages:

1. Block Alignment

    - match blocks using similarity scoring (token overlap / cosine similarity)
2. Intra-block Diff
    - apply text diff (e.g., diff-match-patch) within aligned blocks

This two-stage approach:

- reduces false positives from layout changes
- preserves semantic grouping
- improves alignment stability

Output Formatter

The output will strictly follow the existing JSON schema of the web-monitoring-diff.
This ensures compatibility with the frontend.

Scope and Limitations

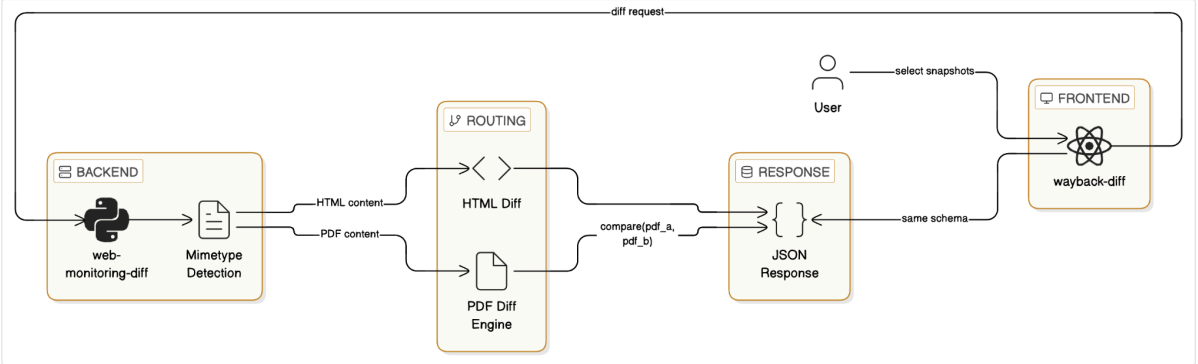
The system is designed to handle a wide range of PDFs, including text-based and moderately complex layout documents.

However, due to the inherent limitations of the PDF format:

- complex tables may not be perfectly reconstructed
- OCR-based diffs may contain noise
- graphical content is not semantically diffed

The focus is on providing robust and meaningful diffs for common real-world documents, with graceful degradation for complex cases.

Integration and Packaging Strategy



Independent Package

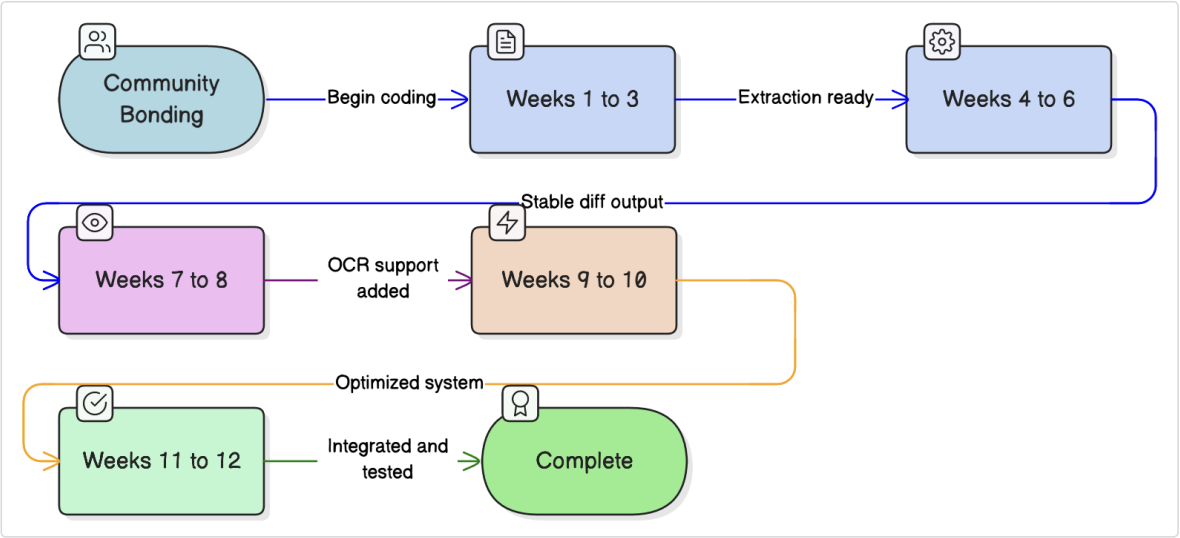
The system will be implemented as a pip-installable Python package:

- stateless and reusable
- importable by web-monitoring-diff
- not a standalone service

Integration Flow

1. web-monitoring-diff fetches captures
2. detects content type
3. routes request
4. JSON returned to frontend
5. wayback-diff renders result

Timeline (12 Weeks)



Community Bonding

- understand web-monitoring-diff
- study Wayback diff flow
- Finalize system design (analyzer, extraction, block representation)
- Set up development and testing environment

Weeks 1–3:

- Implement PDF Analyzer (text density, layout signals)
- Build adaptive extraction pipeline (PyMuPDF, pdfminer.six)
- Normalize extraction output into unified format
- Implement basic normalization and initial block segmentation
- Generate initial diff output for simple PDFs

Milestone: End-to-end pipeline working on text-based PDFs

Weeks 4–6:

- Improve reading order reconstruction (column detection)
- Enhance block segmentation and layout cleanup
- Implement block alignment and intra-block diff

- Ensure compatibility with Wayback JSON schema

Milestone: Stable and meaningful diff output

Weeks 7–8

- Integrate Tesseract OCR for scanned PDFs
- Connect OCR with analyzer-based triggering
- Normalize OCR output and integrate into pipeline

Milestone: Support for scanned PDFs (best-effort)

Weeks 9–10

- Optimize performance (page-wise processing, limits)
- Handle edge cases:
    - multi-column layouts
    - large PDFs
    - corrupted/unsupported files
- Improve diff accuracy and reduce false positives

Milestone: Robust system across diverse PDFs

Weeks 11–12

- Integrate with web-monitoring-diff via MIME-based routing
- Add unit and integration tests
- Build CLI tool for standalone usage
- Complete documentation

Milestone: Fully integrated, tested, and documented system

Risk Mitigation

- **OCR Performance Overhead**
  - OCR (Tesseract) is significantly slower than text-based extraction.
  - It will only be triggered when the analyzer detects extremely low text density or extraction failure.
  - OCR will be applied selectively at the **page level**, not the entire document when possible.
  - Timeouts and page limits will be enforced to prevent excessive processing time.
- **Large PDFs (Memory & Time Constraints)**
  - Large PDFs can cause high memory usage and slow processing.
  - Processing will be done in a **page-wise streaming manner** instead of loading the full document.
  - For the MVP, a configurable page limit (e.g., first 50 pages) will be applied.
  - If truncation occurs, it will be clearly indicated in the output.
- **Layout Complexity (Multi-column, Tables)**
  - Complex layouts can disrupt reading order and block segmentation.
  - The system will use **x-coordinate clustering** to detect multi-column layouts.
  - Reading order will be reconstructed using spatial heuristics.
  - Tables will be handled as **grouped blocks** rather than attempting full structural reconstruction to ensure stability.
- **Extraction Inconsistency Across PDFs**
  - Different extraction tools may produce inconsistent outputs.
  - A **PDF Analyzer** will select extraction strategies deterministically using:
    - text density
    - fragmentation score

        - layout complexity
    - All outputs will be converted into a **unified intermediate format** to ensure consistency downstream.
- **OCR Noise and Inaccurate Text**
    - OCR can introduce errors such as incorrect characters or missing text.
    - Post-processing will include:
        - whitespace normalization
        - basic noise cleanup
    - Block-level diffing reduces sensitivity to minor OCR errors by focusing on semantic units instead of characters.
- **Incorrect Block Segmentation**
    - Imperfect block grouping can affect alignment.
    - Segmentation will use multiple signals:
        - spacing
        - alignment
        - positional consistency
    - The two-stage diff (block alignment + intra-block diff) helps recover correct matches even if segmentation is imperfect.
- **Corrupted or Unsupported PDFs (e.g., encrypted files)**
    - Encrypted or malformed PDFs will be detected during validation or analyzer stage.
    - Such files will be handled gracefully with clear error messages instead of pipeline failure.
- **Performance Variability Across Document Types**
    - Different PDF types (text-heavy vs scanned) have different processing costs.
    - The analyzer ensures lightweight methods (PyMuPDF) are used by default.
    - Heavy operations (OCR) are only used when necessary, optimizing average-case performance.

Evaluation

The system will be evaluated using:

- Reduction in false positives compared to naive text extraction diff
- Accuracy of block alignment (semantic grouping consistency)
- Stability of JSON output across different PDF layouts
- Processing time per document (seconds per page)

Baseline:
Naive text extraction + direct diff

Success Criteria:

- Significant reduction in noisy diffs caused by layout artifacts
- Consistent and schema-compatible JSON output
- Efficient processing for medium-sized PDFs (e.g., <2 seconds per page)

Testing Strategy

- unit tests for normalization, extraction
- integration tests with real PDFs
- edge cases:
    - scanned PDFs
    - layout-heavy documents
    - large files

Deliverables

- pip-installable Python package
- PDF diff engine
- CLI tool
- test suite
- documentation

About Me

I have experience in backend development and system design, with a focus on building practical and robust systems.

I have contributed to the web-monitoring-diff project by improving the HTML diff pipeline, including implementing case-insensitive comparison while preserving correctness and compatibility. This involved working with tokenization and diff logic, giving me a strong understanding of how the Wayback diff system operates. PR#245

I have also developed a prototype PDF diff system, exploring extraction, normalization, and diffing strategies, which forms the foundation for this project.

Final Statement

This project focuses on building a robust and integration-ready PDF diff system using analyzer-guided extraction and block-based comparison. By prioritizing structural normalization, deterministic behavior, and compatibility with existing infrastructure, it aims to extend the Wayback Machine's diff capabilities to PDFs in a practical and scalable manner.